1.5em 0pt

Amortized Analysis & Greedy Algorithm

CSE 4809: Algorithm Engineering
Lecture 05

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

May 19, 2026

Amortized Analysis: The Core Concept

Amortization

Averaging the time required to perform a *sequence* of operations over all operations performed.

👉 **Key Guarantee:** Unlike average-case analysis, there is no probability involved. It provides a **worst-case bound** on the actual cost of the entire sequence. In other words, average performance of each operation in the worst case

Analogy

Spreading a monthly gym membership fee (60\$) over 30 days (2\$/day) added to the daily usage fee(3\$).

👉 **Algorithm Insight:** Insight from amortized analysis can help optimize the design of data structures.

Why Amortized Analysis?

👉 Amortized analysis is a form of asymptotic analysis.

👉 Used instead of a simple worst-case analysis of individual operations. Why:

- **Avoiding Overly Pessimistic Bounds:** Standard asymptotic analysis often bounds a sequence by taking the worst-case cost of a single operation and multiplying it by the number of operations (n). Example *Multipop Stack*
- **Sequence-Level Accuracy:** Standard analysis treats each operation as an independent event. Amortized analysis recognizes that expensive operations change the state of the data structure in a way that prevents another expensive operation from occurring again for a long time.
- **Predictable Long-Term Performance:** Knowing the absolute worst-case of one operation is often less important than knowing that the entire task will complete within a predictable timeframe

Method 1: Aggregate Analysis I

Concept

Show that for all n, a sequence of n operations takes $T(n)$ worst-case time in total. The amortized cost per operation is $T(n)/n$

Example: Multipop Stack

👉 `PUSH(S, x)`, `POP(S)`: $O(1)$ cost.

👉 `MULTIPOP(S, k)`: Pops k elements, cost is $O(\min(s, k))$, s is the size of the stack.

Wrost Case Analysis:

👉 Consider n, `Push`, `Pop` and `Multipop` operation.

- Worst case of `Multipop` is $O(n)$.
- So n sequence operation cost is $O(n^2)$.

Method 1: Aggregate Analysis II

Aggregate Analysis: MULTIPOP

Any sequence of n `PUSH`, `POP`, and `MULTIPOP` operations on an initially empty stack has an upper bound on its cost of $O(n)$.

Why

- 👉 Any object cannot be popped from the stack unless it was first pushed.
- 👉 So, the number of times that `POP` can be called on a nonempty stack, including calls within `MULTIPOP`, is at most the number of `PUSH` operations, which is at most n
- 👉 For any value of n, any sequence of n `PUSH`, `POP`, and `MULTIPOP` operations takes a total of $O(n)$ time. On average $O(n)/n = O(1)$.

Method 1: Aggregate Analysis III

Example: Incrementing a Binary Counter

- A k-bit binary counter that counts upward from 0.
- An array $A[0 : k-1]$ of bits represents the counter.
- A number x stored in A is defined by $x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$
- Initially $x = 0$, i.e $A[i] = 0 \quad$ for, $\quad i = 0, 1, 2 \ldots k-1$

Worst Case Analysis:

- A single execution of `INCREMENT` takes ,$\Theta(k)$ time in the worst case. (All k bit was 1).
- Thus, a sequence of n `INCREMENT` operationson an initially zero counter takes $O(nk)$ time in the worst case.
- But it is not **tight** bound. **Why**?

Method 1: Aggregate Analysis IV

```
INCREMENT(A, k)
1  i = 0
2  while i < k and A[i] == 1
3      A[i] = 0
4      i = i + 1
5  if i < k
6      A[i] = 1
```

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Figure: Increment Procedure and example of 16 bit counter

Method 1: Aggregate Analysis V

Aggregate Analysis: Incrementing Binary Counter

A sequence of n `INCREMENT` operations on an initially zero counter takes $O(n)$ time in the worst case. So average cost is $O(n)/n = O(1)$.

Why?

Method 1: Aggregate Analysis VI

Amortized Analysis

- A single call of `INCREMENT` might flip all k bits, not all bits flips upon each call.
- $A[0]$ does flip each time `INCREMENT` is called i.e n times.
- $A[1]$ flips every 2nd times, i.e. $\lfloor n/2 \rfloor$ time on n calls.
- $A[2]$ flips every 4rth times, i.e. $\lfloor n/4 \rfloor$ time on n calls.
- $A[i]$ flips every 2^i time, i.e. $\lfloor n/2^i \rfloor$ time on n calls for $i = 0, 1, 2, \ldots k-1$
- Total number of flips on n Increment Procedure call is

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

- On average $O(n)/n = O(1)$.

Method 2: The Accounting Method I

Concept

Assign differing charges (*amortized costs*) to different operations. Some opera are overcharged to store **credit** on specific objects.

Rule

The total credit ($\sum \hat{c}_i - \sum c_i$) must remain **non-negative** at all times to ensure the amortized sum upper-bounds the actual sum.

Method 2: The Accounting Method II

Binary Counter

- **Actual cost**: Number of bits flipped.
- Amortized charge for setting a bit from 0 to 1: \$2 (\$1 to set, \$1 credit).
- Amortized charge for resetting a bit from 1 to 0: \$0 (uses stored credit \$1).
- **Result:** Every `INCREMENT` sets at most one bit, so amortized cost is \$2 i.e. $O(1)$.

Classwork: MULTIPOP

Method 3: The Potential Method

Concept

Prepaid work is viewed as **potential energy** (Φ) of the data structure as a whole instead of prepaid cost of each operations.

Amortized Cost Formula

Intiial Data Structure D_0 under going n sequential operatons. For operation $i = 1, 2, \ldots n$ let the actual cost be c_i. D_i is the state of data structure after i^{th} operations. $\Phi(D_i)$ is the potential energy of D_i.

- **Amortized cost:** $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$
- **Total Cost:** $\sum \hat{c}_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$.
- **Requirement:** Usually $\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ for all i
- **Intuition:** If $\Delta\Phi > 0$, we overcharge. If $\Delta\Phi < 0$, the potential pays for the operation

Dynamic Tables: Expansion I

The Challenge

Pre-allocation of table size is often impossible or wasteful when you don't know the actual number of elements to be stored.

Load Factor (α)

The ratio of stored elements and table size ($\alpha(T) = num/size$)

Dynamic Tables: Expansion II

Amortized Analysis

- **Operations of Table:** `TABLE -INSERT` and `TABLE-DELETE`
- **Table Expansion:** While table is full and insertion is made.
- T is an object representing the table. $T.table$ points the memory block of T. $T.num$ number of item in the table T. $T.size$ total size of the table.

```
TABLE-INSERT(T, x)
 1  if T.size == 0
 2      allocate T.table with 1 slot
 3      T.size = 1
 4  if T.num == T.size
 5      allocate new-table with 2 · T.size slots
 6      insert all items in T.table into new-table
 7      free T.table
 8      T.table = new-table
 9      T.size = 2 · T.size
10  insert x into T.table
11  T.num = T.num + 1
```

Figure: Insertion Procedure

Dynamic Tables: Expansion III

Amortized Analysis

Actual cost c_i defined as

$$c_i = \begin{cases} i & \text{if } i-1 \text{is a power of 2} \\ 1 & \text{otherwise} \end{cases}$$

Total cost over n insertion

$$\begin{aligned} \sum_{i=1}^{n} c_i &\leq n + \sum_{j=0}^{\lfloor lg\ n \rfloor} 2^j \\ &= n + 2n \\ &= 3n \end{aligned}$$

Average cost $O(3n)/n = O(3) = O(1))$

Dynamic Tables: Contraction and Thrashing

The Goal

Save space by shrinking when many items are deleted

- **The Thrashing Trap:**
  - Doubling at $\alpha = 1$ and halving at $\alpha = 1/2$.
  - Alternating *insert, delete, delete, insert...* near the boundary triggers repeated $\Theta(n)$ work.
- **The Solution:** Wait until the table is **1/4 full** before halving it.
- **Result:** This strategy guarantees the load factor stays between $1/4$ and 1, and the amortized cost remains $O(1)$.

What is a Greedy Algorithm?

Core Idea

A greedy algorithm always makes the choice that looks **best at the moment** — a *locally optimal* choice — in the hope of finding a *globally optimal* solution.

- Algorithms for **optimization problems** go through a sequence of steps, with choices at each step.
- Dynamic programming considers *all* choices; greedy picks *one* — the locally best one.
- Greedy is simpler & more efficient than DP when applicable.

Important Caveat

Greedy algorithms do **not** always yield optimal solutions. We must **prove** that greedy works for each specific problem.

Analogy

Imagine navigating a maze: a greedy navigator always turns toward the direction that *looks closest* to the exit — sometimes it works perfectly, sometimes it leads to dead ends.

🤝 Reference & Q&A

Reference: Introduction to Algorithms, CLRS

- 👉 **Chapter 11**
- 👉 **Chapter 16**

Thank You!

👋 Thank you for your time and attention. 👋

🗨️ Any questions or clarifications. 💭